

UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE LOS VALLES

Lic. En Tecnologías de la Información.



Introducción a la programación web con Python - MySQL- Flask

Desarrollo de Bases de Datos para Web

Alumno: Sergio Alejandro Rodríguez Hermosillo.

Profesor: Adolfo Castillo Chavarrín.

Viernes, 25 de septiembre de 2026 de 2026.

En el mundo del desarrollo de aplicaciones web modernas, la interacción entre la interfaz grafica que ve el usuario final, la logia del servidor que procesa las peticiones y el almacenamiento persistente de datos es un pilar fundamental. En esta práctica desarrolle una aplicación web completa basada en el patrón CRUD (Crear, Leer, Actualizar y Eliminar) para la gestión de un catálogo de videojuegos.

La aplicación utiliza Python como lenguaje principal junto con el microframework Flask para el enrutamiento web y control de peticiones HTTP. Para la capa de persistencia se integro un servidor de base de datos **MySQL/MariaDB** administrado mediante la librería pymysql. La interfaz de usuario fue construida utilizando plantillas **HTML** dinamizadas con el motor de renderizado Jinja2 y estilizadas con el framework **CSS Bulma**.

Cada registro dentro del sistema representa un videojuego definido por cuatro atributos esenciales: **id** (Clave primaria de tipo entero auto incrementable), **nombre** (Cadena de texto para la identificación del título), **descripción** (Texto descriptivo sobre el genero o trama del juego), **precio** (Valor numerico decimal para el costo del titulo).

Para garantizar la mantenibilidad y modularidad del proyecto, el código se organizo bajo una arquitectura desacoplada en tres modulos principales de Python: **conexion.py** (gestión de conexiones), **controlador_juegos.py** (consultas SQL y lógica de datos) y **main.py** (servidor web y rutas Flask). Ademas, se utlizio la carpeta **templates/** para albergar la vista maestra y las vistas secundarias.

El propósito central del ejercicio fue comprender y poner en practica el flujo completo de transferencia de datos a través de las distintas capas de una aplicación web (Frontend, Backend y Database).

Objetivos específicos:

1. Diseñar e implementar la estructura relacional de la base de datos en MySQL.
2. Configurar la conexión segura desde Python hacia MySQL utilizando un gestor de cursores.
3. Consultar y desplegar los registros almacenados en una tabla HTML interactiva mediante peticiones **GET**.
4. Procesar formularios de alta información enviando datos por el método **POST**.
5. Implementar mdoificacion de datos existentes utilizando identificadores dinámicos y formularios precargados.
6. Aplicar la eliminación controlada de registros filtrando por clave primaria.
7. Practicar el uso de sentencias SQL parametrizada para prevenir vulnerabilidades de Inyeccion SQL.

En el desarrollo comenzamos con la preparación del servidor local de base de datos MySQL a través de XAMPP. Cree la base de datos llamada juegos y ejecute la instrucción **DDL** (Data Definition Language) para construir la tabla correspondiente.

```
Setting environment for using XAMPP for Windows.
cd c:\xampp
mysql -u root
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password: NO)

cd c:\xampp
mysql -u root -p
Enter password: *****
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 9
Server version: 10.4.32-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\q' to quit the current input statement.

MariaDB [(none)]> CREATE DATABASE juegos;
Query OK, 1 row affected (0.005 sec)

MariaDB [(none)]> use juegos;
Database changed
MariaDB [juegos]> CREATE TABLE juegos(id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(255) NOT NULL, descripcion VARCHAR(255) NOT NULL, precio DECIMAL(9,2) NOT NULL);
Query OK, 0 rows affected (0.017 sec)

MariaDB [juegos]> SHOW TABLES;
+-----+
| Tables_in_juegos |
+-----+
| juegos            |
+-----+
1 row in set (0.002 sec)

MariaDB [juegos]> DESCRIBE juegos;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| id     | bigint(20) unsigned | NO | PRI | NULL | auto_increment |
| nombre | varchar(255)        | NO |     | NULL |                 |
| descripcion | varchar(255)      | NO |     | NULL |                 |
| precio | decimal(9,2)        | NO |     | NULL |                 |
+-----+-----+-----+-----+-----+
4 rows in set (0.023 sec)

MariaDB [juegos]> INSERT INTO juegos(nombre, descripcion, precio)
-> VALUES
-> ("MINECRAFT", "Explora un mundo abierto en el que tu creas tus propias herramientas, sobrevives la noche llena de criaturas que intentaran acurrir contigo", 245.00),
-> ("Counter Strike Source", "Un shooter clasico que definio toda una era competitiva, con la ultima actualizacion no se queda atras con los graficos", 85.00),
-> ("Need for Speed", "Explora la ciudad Redview y enfrentate a tus rivales, pero quien dijo que sera facil, una horda de policia espera conseguir tu captura", 100.00);
Query OK, 3 rows affected (0.050 sec)
Records: 3  Duplicates: 0  Warnings: 0

MariaDB [juegos]> SELECT * FROM juegos;
+----+-----+-----+-----+
| id | nombre | descripcion | precio |
+----+-----+-----+-----+
| 1  | MINECRAFT | Explora un mundo abierto en el que tu creas tus propias herramientas, sobrevives la noche llena de criaturas que intentaran acurrir contigo | 245.00 |
| 2  | Counter Strike Source | Un shooter clasico que definio toda una era competitiva, con la ultima actualizacion no se queda atras con los graficos | 85.00 |
| 3  | Need for Speed | Explora la ciudad Redview y enfrentate a tus rivales, pero quien dijo que sera facil, una horda de policia espera conseguir tu captura | 100.00 |
+----+-----+-----+-----+
3 rows in set (0.002 sec)

MariaDB [juegos]>
```

Posteriormente, configure el entorno virtual de desarrollo en Python para aislar las dependencias del proyecto. La conexión a la base de datos se aislo en el archivo **conexion.py** para evitar la duplicación de credenciales en el código.

```
1 # =====
2 # IMPORTACION DE LIBRERIAS
3 # =====
4 import pymysql
5
6 # =====
7 # FUNCION PARA OBTENER LA CONEXION A LA BASE DE DATOS
8 # =====
9 def obtener_conexion():
10     # Cada llamada crea una conexión PyMySQL usando la configuración local del proyecto.
11     return pymysql.connect(
12         host='localhost',
13         user='root',
14         password='15054505',
15         db='juegos'
16     )
```

Para listar los videojuegos, construí la función **obtener_juegos()** en el archivo **controlador_juegos.py**. La función abre la conexión, ejecuta una sentencia **SELECT**, recupera todas las filas mediante **cursor.fetchall()** y retorna una lista de tuplas antes de cerrar la conexión.

```
20 # =====
21 # FUNCION PARA OBTENER TODOS LOS JUEGOS DE LA BASE DE DATOS
22 # =====
23 def obtener_juegos():
24     """ Obtiene todos los juegos de la base de datos. """
25     conexion = obtener_conexion()
26     juegos = []
27     with conexion.cursor() as cursor:
28         # fetchall devuelve todas las filas para mostrarlas en la tabla.
29         cursor.execute("SELECT id, nombre, descripcion, precio FROM juegos")
30         juegos = cursor.fetchall()
31     conexion.close()
32     return juegos
```

En **main.py**, se registran las rutas **"app.route("/")** y **@app.route("/juegos")**, las cuales invocan al controlador y pasan la lista de registros a la plantilla mediante **render_template("juegos.html", juegos=juegos)**.

La interfaz utiliza herencia de plantillas mediante Jinja2. El archivo **maestra.html** define la estructura HTML5 y la barra de navegación.

```
6 <!DOCTYPE html>
7 <html lang="es">
8 <head>
9     <meta charset="UTF-8">
10    <meta http-equiv="X-UA-Compatible" content="IE=edge">
11    <meta name="viewport" content="width=device-width, initial-scale=1.0">
12    <!-- Las plantillas hijas reemplazan estos bloques con el título y el contenido de cada vista. -->
13    <title>CRUD MySQL con Flask - {% block titulo %}{% endblock %}</title>
14    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bulma@0.9.2/css/bulma.min.css">
15 </head>
```

El archivo **juegos.html** extiende de esta estructura y recorre los registros usando una directiva de ciclo.

```
1 {% for juego in juegos %}
2 <tr>
3     <td>{{ juego[1] }}</td>
4     <td>{{ juego[2] }}</td>
5     <td>{{ juego[3] }}</td>
6     <td><a href="{{ url_for('editar_juego', id=juego[0]) }}" class="button is-info">Editar</a></td>
7 </tr>
8 {% else %}
9 <tr><td colspan="5">No hay juegos registrados.</td></tr>
10 {% endfor %}
```

La inserción de un videojuego consta de dos etapas: la visualización del formulario y el procesamiento de los datos.

La ruta `/agregar_juego` renderiza el archivo `agregar_juego.html`. El formulario captura el nombre, descripción y precio, utilizando la propiedad `method="post"` para enviar información hacia la ruta `/guardar juego`.

```
1 <form action="{{ url_for('guardar_juego') }}" method="post">
2   <input required type="text" class="input" placeholder="Nombre" name="nombre" id="nombre">
3   <input required type="text" class="input" placeholder="Descripción" name="descripcion" id="descripcion">
4   <input required type="number" step="any" class="input" placeholder="Precio" name="precio" id="precio">
5   <button type="submit" class="button is-success mt-2">Guardar</button>
6 </form>
```

En el backend, Flask lee las variables del formulario mediante el objeto `request.form`. Posteriormente, se ejecuta la función `insertar_juego` en el contador.

```
9 def insertar_juego(nombre, descripcion, precio):
10     """ Inserta un nuevo juego en la base de datos. """
11     conexion = obtener_conexion()
12     with conexion.cursor() as cursor:
13         # Los marcadores %s separan los datos del SQL y evitan concatenar valores en la consulta.
14         cursor.execute("INSERT INTO juegos(nombre, descripcion, precio) VALUES (%s, %s, %s)",
15                        (nombre, descripcion, precio))
16     # Las escrituras requieren confirmación para persistir los cambios.
17     conexion.commit()
18     conexion.close()
```

Es importante destacar el uso de marcadores `%s`. Estos actúan como sentencias preparadas (prepared statements), permitiendo que el motor de base de datos trate los valores como datos puros y no como código ejecutable, neutralizando ataques de inyección SQL. Después de confirmar la transacción con `conexión.commit()`, el servidor responde con un redireccionamiento `redirect("/juegos")`.

El proceso de modificación requirió pasar el identificador del registro a través de la URL. Al presionar el botón "Editar", se navega a la ruta dinámica `/formulario_editar_juego/<int:id>`.

El servidor procesa esta petición obteniendo los datos del juego específico con la función `obtener_juego_por_id(id)` que utiliza `cursor.fetchone()`.

```
49 def obtener_juego_por_id(id):
50     """ Obtiene un juego de la base de datos por su ID. """
51     conexion = obtener_conexion()
52     juego = None
53     with conexion.cursor() as cursor:
54         # fetchone devuelve una sola fila, que se usa para rellenar el formulario de edición.
55         cursor.execute(
56             "SELECT id, nombre, descripcion, precio FROM juegos WHERE id = %s", (id,))
57         juego = cursor.fetchone()
58     conexion.close()
59     return juego
```

Los datos recuperados se envían a **editar_juego.html**, donde se rellenan automáticamente los atributos **value** de las entradas de texto. Un punto crítico en la implementación fue la inclusión de un campo de entrada oculto.

```
1 <form action="{{ url_for('actualizar_juego') }}" method="post">
2   <input type="hidden" name="id" value="{{ juego[0] }}"> {#justo aqui#}
3   <input value="{{ juego[1] }}" required type="text" class="input" name="nombre">
4   <input value="{{ juego[2] }}" required type="text" class="input" name="descripcion">
5   <input value="{{ juego[3] }}" required type="number" step="any" class="input" name="precio">
6   <button type="submit" class="button is-success mt-2">Guardar</button>
7 </form>
```

Este campo garantiza que el identificador viaje de regreso al servidor en la petición **POST** hacia **/actualizar_juego** sin ser alterado visualmente por el usuario. La función del controlador ejecuta la instrucción de actualización.

```
64 def actualizar_juego(nombre, descripcion, precio, id):
65     """ Actualiza un juego en la base de datos por su ID. """
66     conexion = obtener_conexion()
67     with conexion.cursor() as cursor:
68         # Actualiza los campos editables únicamente para el juego indicado por su ID.
69         cursor.execute("UPDATE juegos SET nombre = %s, descripcion = %s, precio = %s WHERE id = %s",
70                        (nombre, descripcion, precio, id))
71     conexion.commit()
72     conexion.close()
```

La eliminación de un videojuego se implementó mediante un formulario individual por cada fila de la tabla para evitar borrados accidentales mediante peticiones **GET** en la **URL**. El formulario envía el **ID** por método **post** a la ruta **/eliminar_juego**.

```
16 <form action="{{ url_for('eliminar_juego') }}" method="post">
17   <input type="hidden" name="id" value="{{ juego[0] }}">
18   <button type="submit" class="button is-danger">Eliminar</button>
19 </form>
```

El controlador ejecuta la eliminación condicionada exclusivamente por la clave primaria:

```
37 def eliminar_juego(id):
38     """ Elimina un juego de la base de datos por su ID. """
39     conexion = obtener_conexion()
40     with conexion.cursor() as cursor:
41         # La condición limita el borrado al registro cuyo ID recibió la ruta.
42         cursor.execute("DELETE FROM juegos WHERE id = %s", (id,))
43     conexion.commit()
44     conexion.close()
```

Durante el desarrollo de la practica me vi envuelto en varios errores y retos técnicos que requirieron una depuración.

Error HTTP 404 (Not Found) en la raíz: Al iniciar el servidor Flask e ingresar a http://127.0.0.1:5000/, el navegador devolvía un error 404. Tras revisar el archivo main.py, identifiqué que solo existía el decorador @app.route("/juegos"). Lo solucioné agregando el decorador múltiple @app.route("/") encima de la función del listado.

Incompatibilidad en peticiones INSERT: Inicialmente intenté enviar el campo id dentro de la consulta INSERT pasando una cadena vacía. Esto provocaba un error de sintaxis en **MySQL** debido a que la columna es entera y de tipo **AUTO_INCREMENT**. La solución fue omitir la columna id de la lista de campos en la sentencia SQL para que el motor asigne la secuencia de manera automática.

Persistencia de cambios en MySQL: Al probar las primeras inserciones, no se registraban errores, pero la tabla en la base de datos permanecía vacía. Identifiqué que la librería **pymysql** no realiza un guardado automático (**autocommit**), por lo que fue necesario agregar explícitamente **conexion.commit()** después de cada instrucción **INSERT**, **UPDATE** y **DELETE**.

Acceso a elementos por índice en Jinja2: Debido a que **pymysql** devuelve las filas como tuplas de Python, en las plantillas HTML se debía acceder a cada dato mediante su índice numérico (**juego[0]** para **ID**, **juego[1]** para **nombre**, etc.). Tuve que mapear correctamente estos índices en el archivo juegos.html para evitar mostrar datos cruzados.

A lo largo de esta practica me siento significativamente fortalecido ya que mi comprensión sobre el desarrollo de software web del lado del servidor (**backend**) y su integración con bases de datos relacionales. Me quedo claro la importancia de GET exclusivamente para la recuperación de información que no altera el estado del servidor, mientras que POST se debe emplear para el envío de datos sensibles o modificaciones en la base de datos. Comprendí que las operaciones de modificación de datos requieren una confirmación explícita (**commit()**) para asegurar la integridad de la base de datos de acuerdo con los principios ACID.

Además, Entendí el mecanismo mediante el cual los controladores separan la lógica de la consulta SQL de los valores introducidos por los usuarios mediante tuplas de parámetros, eliminando el riesgo de Inyección **SQL**. Mejore mis habilidades en el uso del motor Jinja2 me permitió comprender cómo la herencia mediante bloques (**{% block contenido %}**) reduce la duplicación de código en el desarrollo **frontend**.

En futuros proyectos el sistema dependerá de la validación básica del navegador **HTML5**. Es necesario implementar validación en el servidor para verificar tipos de datos y rangos antes de interactuar con la base de datos. Es por eso que se requiere implementar bloques **try-except-finally** alrededor de las conexiones a la base de datos para gestionar caídas de servidor o errores de red sin que la aplicación falle. Además, las credenciales de la base de datos deben estar escrita en un archivo independiente para mejorar la seguridad y el acceso a ella, me queda claro que en proyectos profesionales se deben, manejar mediante variables de entorno.

En conclusión, se ha logrado con éxito todos los objetivos planteados al inicio y al final de esta práctica. Logre construir un sistema funcional de administración de videojuegos capaz de realizar las cuatro operaciones fundamentales del modelo **CRUD** mediante una interfaz grafica web limpia y responsiva.

Durante el desarrollo del sistema demostré la efectividad de utilizar Python y Flask para la creación rápida de prototipos web, así como la robustez de **MySQL** para el almacenamiento relacional. La separación modular del código en archivos independientes, que no solo facilito la construcción paso a paso, sino que demostró ser una practica muy indispensable para el mantenimiento del software.